

Enterprise Test Automation Framework Architecture Document

Document Control

Field	Value
Project Name	<Project Name>
Client Name	<Client Name>
Prepared By	<Your Name / QA Team>
Role	Senior Test Architect
Version	1.0
Date	<DD-MMM-YYYY>
Approval	<Stakeholder Name>

1. Objective

This document defines the architecture and design of a **Hybrid Test Automation Framework** for:

- **Web UI Automation** using Playwright
 - **API Automation** using RestAssured
 - Designed for **scalability, maintainability, and enterprise reuse**
-

2. Scope

Area	Coverage
UI Testing	Functional & Regression
API Testing	Functional validation
Execution	Local (QA Environment)
Data Strategy	Static + Dynamic
Reporting	Allure + Logs

3. Architecture Overview

◆ Key Principles

- Layered Architecture
 - Separation of Concerns
 - Reusability & Modularity
 - Config-driven execution
-

✿ 4. High-Level Architecture Layers

Layer	Responsibility
Test Layer	Test cases (UI + API)
UI Layer	Page Object Model (Playwright)
API Layer	Client + Service Layer (RestAssured)
Core Layer	Base classes, config
Utilities	Logging, helpers
Data Layer	JSON/CSV + dynamic
Reporting	Allure, logs, screenshots

📁 5. Directory Structure

◆ Root Structure

automation-framework/

├─ ui-automation/

├─ api-automation/

├─ common/

├─ config/

├─ test-data/

├─ reports/

├─ logs/

- └─ scripts/
- └─ docs/
- └─ README.md

◆ UI Automation (Playwright)

- ui-automation/
 - └─ tests/
 - | └─ functional/
 - | └─ regression/
 - |
 - └─ pages/
 - └─ fixtures/
 - └─ utils/
 - └─ test-data/
 - └─ config/
 - └─ hooks/
 - └─ reports/

◆ API Automation (RestAssured)

- api-automation/
 - └─ tests/
 - └─ clients/
 - └─ services/
 - └─ models/
 - └─ utils/
 - └─ validators/
 - └─ config/

└─ reports/

6. Execution Flow

UI Flow

Test → Fixture → Page → Utility → Assertion → Report

API Flow

Test → Service → Client → Request → Validator → Report

7. Test Data Strategy

Type	Approach
------	----------

Static	JSON / CSV
--------	------------

Dynamic	Runtime generation
---------	--------------------

Usage	Hybrid
-------	--------

8. Reporting & Logging

Feature	Tool
---------	------

HTML Reports	Allure
--------------	--------

Screenshots	On failure
-------------	------------

Videos	Playwright
--------	------------

Logging	log4j + custom logger
---------	-----------------------

9. Configuration Management

config/

└─ qa.properties

└─ env.ts

- Supports future:
 - UAT

- Prod
- Multi-env execution

10. Naming Conventions

Component Naming

Test Files *.spec.ts, *Test.java

Pages LoginPage.ts

Services UserService.java

Data Files userData.json

11. Future Enhancements

- CI/CD integration (Jenkins/GitHub Actions)
- Parallel execution scaling
- Docker support
- Test tagging strategy
- Retry mechanism

12. Senior Test Automation Manager Review & Verdict

Now evaluating this as I would in a **real enterprise governance review**:

✅ What's Excellent (Enterprise Strengths)

✓ 1. Strong Architectural Foundation

- Clear separation: UI vs API vs Core
- Layered approach aligns with **industry best practices**

👉 This ensures:

- Maintainability
- Team scalability

- Clean ownership boundaries
-

✓ 2. API Service + Client Layer (Very Strong)

This is often missing in mid-level frameworks.

👉 Your design includes:

Test → Service → Client → API

✓ Promotes:

- Reusability
 - Business abstraction
 - Cleaner test cases
-

✓ 3. Hybrid Data Strategy

- Static + Dynamic = **high stability + flexibility**

✓ Reduces:

- Test flakiness
 - Data dependency issues
-

✓ 4. Reporting & Debugging Readiness

- Allure + logs + screenshots + video

👉 This is **production-grade observability**

⚠️ Key Risks / Improvement Areas

⚠️ 1. Dual Technology Stack (Major Consideration)

Area Tech

UI TypeScript

API Java

👉 Risk:

- Increased onboarding effort
- Skill dependency
- Maintenance overhead

✓ Recommendation:

- Either:
 - Define **separate ownership teams**, OR
 - Move API to Playwright (long-term simplification)
-

⚠ 2. No CI/CD Integration (Critical Gap)

Even if currently local:

👉 Enterprise expectation = **automated pipelines**

✓ Add:

/ci/

└─ pipeline.yml

⚠ 3. Missing Execution Governance

No:

- Tagging strategy
- Suite control
- Execution filters

👉 Risk:

- Regression chaos
- Slow execution

✓ Recommendation:

- Introduce tags:
 - @smoke
 - @regression
 - @api

- @ui

⚠️ 4. No Flaky Test Handling

👉 Enterprise frameworks MUST include:

- Retry logic
- Stability tracking

✓ Add:

- Playwright retries
- TestNG Retry Analyzer

⚠️ 5. Environment Scalability Limitation

Currently:

- Only QA

👉 Risk:

- Hardcoded configs later

✓ Recommendation:

config/

├─ qa/

├─ uat/

└─ prod/

📊 Final Scorecard

Area	Rating
Architecture	★★★★★ (5/5)
Scalability	★★★★☆ (4/5)

Area	Rating
Maintainability	★★★★★ (5/5)
DevOps Readiness	★★☆☆☆ (2/5)
Overall	8.5 / 10

🔍 Final Verdict (Manager Perspective)

👉 “This is a solid enterprise-ready framework foundation with strong design principles. With CI/CD integration, execution governance, and stack alignment, it can evolve into a best-in-class automation platform.”
